

Khant Thu Aung/ 105292912

9.2C / Iteration 7

Path.cs

namespace SwinAdventure;

public class Path : GameObject

{

Location \_destination;

string \_name;

public Path(string[] ids, string name, string desc, Location destination) : base(ids, name, desc)

{

\_name = name;

\_destination = destination;

}

public void PlayerMove(Player p)

{

p.Location = \_destination;

}

public string GetPathName

{

get { return \_name; }

}

//property

```
    public Location Destination
    {
        get { return _destination; }
    }
}
```

MoveCommand.cs

```
namespace SwinAdventure;
```

```
public class MoveCommand : Command
{
    public MoveCommand() :
    base(new string[] { "move", "head", "leave", "go" }){}

    public override string Execute(Player p, string[] text)
    {
        if (text.Length < 2) return "Move where?";

        string direction = text[1];

        Path path = p.Location.GetPath(direction);

        if (path == null)
            return $"There is no path to {direction}";

        path.PlayerMove(p);

        return $"You move {direction} to {p.Location.Name}.\n{p.Location.FullDescription}";
    }
}
```

```
}
```

Location.cs

```
namespace SwinAdventure;
```

```
public class Location : GameObject, IHaveInventory
```

```
{
```

```
    private Inventory _inventory;
```

```
    private List<Path> _paths;
```

```
    public Location(string name, string desc) : base(new string[] { name.ToLower() }, name, desc)
```

```
{
```

```
    _paths = new List<Path>();
```

```
    _inventory = new Inventory();
```

```
}
```

```
public GameObject Locate(string id)
```

```
{
```

```
    if (AreYou(id)) { return this; }
```

```
    return Inventory.Fetch(id);
```

```
}
```

```
public void AddPath(Path path)
```

```
{
```

```
    _paths.Add(path);
```

```
}
```

```
public Path GetPath(string direction)
```

```
{
```

```

        foreach (Path p in _paths)
        {
            if (p.AreYou(direction))
            {
                return p;
            }
        }
        return null;
    }

    public Inventory Inventory { get { return _inventory; } }

    public override string FullDescription
    {
        get
        {
            string exits = "";

            foreach (Path path in _paths)
            {
                exits += $"- {path.FirstID} to {path.Destination.Name}\n";
            }

            return $"You are at {base.Name}\nThis is a {base.FullDescription}\n.In this
{base.Name}, you can see \n{Inventory.ItemList}\nFrom this area, you can go \n{exits}";
        }
    }
}

LookCommand.cs

namespace SwinAdventure;

```

```

public class LookCommand : Command
{

    public LookCommand() : base(new string[] { "look" }) { }

    public override string Execute(Player p, string[] text)
    {
        IHaveInventory container;

        if(text.Length== 1 && text[0].ToLower() == "inventory")
        {
            container = p;

            return LookAtIn("me", container);
        }

        if (text[0].ToLower() != "look")
        {
            return "Error in look input";
        }

        if (text.Length == 1 && text[0].ToLower() == "look")
        {
            return p.Location.FullDescription;
        }

        if (text.Length != 3 && text.Length != 5)
        {
            return "I don't know how to look like that";
        }

        if (text[1].ToLower() != "at")

```

```

{
    return "What do you want to look at?";
}
if (text.Length == 5 && text[3].ToLower() != "in")
{
    return "What do you want to look in?";
}
string itemId = text[2];

if (text.Length == 3)
{
    container = p;
}
else
{
    string containerId = text[4];
    container = FetchContainer(p, containerId);
    if (container == null)
    {
        return $"I can't find the {containerId}";
    }
}
return LookAtIn(itemId, container);
}

```

```

private IHaveInventory FetchContainer(Player p, string containerId)

```

```

{
    GameObject obj = p.Locate(containerId);
    return obj as IHAVEInventory;
}

private string LookAtIn(string thingId, IHAVEInventory container)
{
    GameObject item = container.Locate(thingId);
    if (item == null)
    {
        return $"I can't find the {thingId}";
    }
    return item.FullDescription;
}
}

```

Program.cs

```
namespace SwinAdventure;
```

```
public class Program
```

```

{
    public static void Main()
    {
        string playerName;
        string playerDescription;

        Console.WriteLine("Enter your name, Adventurer!");
        Console.Write("Name -> ");
    }
}

```

```
playerName = Console.ReadLine();
```

```
Console.WriteLine("Now how do you describe yourself, Adventurer!");
```

```
Console.Write("Description -> ");
```

```
playerDescription = Console.ReadLine();
```

```
Location spawnPoint = new("Front Gate", "a Front Gate of the theme park.");
```

```
Player me = new(playerName, playerDescription, spawnPoint);
```

```
me.Location = spawnPoint;
```

```
//locations
```

```
Location ferrisWheel = new("Ferris Wheel", "a ferris wheel area that many people are lining to ride.");
```

```
Location jungleRide = new("Jungle Ride", "a water ride with the vibe full of jungle.\n You can even hear the bird noise from the front.");
```

```
Location hauntedHouse = new("Haunted House", "a really scary haunted house that many screaming voices are coming from there.");
```

```
Location iceCreamShop = new("Ice-cream Shop", "an icecream shop that many parents are lining up for their children.");
```

```
Location boatRace = new("Boat Race", "a boat race where you can participate.\nA race just began.");
```

```
//exits for each location
```

```
//from spawn
```

```
Path spawnEast = new(new string[] { "east", "e"}, "Concrete Path", "a path leading to ferris wheel area", ferrisWheel);
```

```
spawnPoint.AddPath(spawnEast);
```



```
Path ferrisEast = new(new string[] { "east" ,"e"}, "Concerete Path", "a path leading to  
Jungle Ride area", jungleRide);
```

```
Path ferrisWest = new(new string[] { "west","w" }, "Concerete Path", "a path leading back  
to gate. ", spawnPoint);
```

```
ferrisWheel.AddPath(ferrisEast);
```

```
ferrisWheel.AddPath(ferrisWest);
```

```
Path jungleWest = new(new string[] { "west" ,"w"}, "Concerete Path", "a path leading  
back to Ferris Wheel area", ferrisWheel);
```

```
Path jungleNorth = new(new string[] { "north" ,"n"}, "Dirt Path", "a path leading to  
Haunted House area", hauntedHouse);
```

```
jungleRide.AddPath(jungleWest);
```

```
jungleRide.AddPath(jungleNorth);
```

```
Path hauntEast = new(new string[] { "east" ,"e"}, "Small Bridge", "a path leading to Ice  
Cream Shop", iceCreamShop);
```

```
Path hauntSouth = new(new string[] { "south" ,"s" }, "Dirt Path", "a path leading back to  
Jungle Ride Area", jungleRide);
```

```
Path hauntSe = new(new string[] { "southeast" ,"se" }, "Stone Path", "a path leading to  
Boat Racing Area", boatRace);
```

```
hauntedHouse.AddPath(hauntEast);
```

```
hauntedHouse.AddPath(hauntSouth);
```

```
hauntedHouse.AddPath(hauntSe);
```

```
Path iceSouth = new(new string[] { "south","s" }, "Concerete Path", "a path leading to  
Boat Racing Area", boatRace);
```

```
Path iceWest = new(new string[] { "west","w" }, "Small Bridge", "a path leading back to  
Haunted House Area", hauntedHouse);
```

```
iceCreamShop.AddPath(iceSouth);
```

```
iceCreamShop.AddPath(iceWest);
```

```
Path boatNw = new(new string[] { "northwest","nw" }, "Stone Path", "a path leading to  
Haunted House Area", hauntedHouse);
```

```
Path boatNorth = new(new string[] { "north", "n"}, "Concerete Path", "a path leading to  
Ice Cream Area", iceCreamShop);
```

```
boatRace.AddPath(boatNw);
```

```
boatRace.AddPath(boatNorth);
```

```
//items
```

```
Item phone = new(new string[] { "phone", "modern phone" }, "A modern phone", "A  
phone that you have been using for 2 years.");
```

```
Item wallet = new(new string[] { "wallet", "old wallet" }, "An old wallet", "A wallet that  
your dad gifted on your 10th birthday.");
```

```
Bag bag = new(new string[] { "bag", "school bag" }, "A school bag", "A blue school bag  
that you take everywhere you go.");
```

```
Item ring = new(new string[] { "ring", "gold ring" }, "A gold ring", "A high value ring that's  
been passed down to you");
```

```
Item ticket = new(new string[] { "ticket", "entry ticket" }, "An entry ticket", "A ticket to the  
theme park.");
```

```
Item miniFan = new(new string[] { "fan", "small fan" }, "A small fan", "A small fan that can  
make you cool");
```

```
Item coin = new(new string[] { "coin", "bronze coin" }, "A bronze coin", "A bronze coin that  
someone dropped");
```

```
Item flower = new(new string[] { "flower", "yellow flower" }, "A yellow flower", "A yellow  
flower growing near the Jungle Ride");
```

```
Item skull = new(new string[] { "skull", "plastic skull" }, "A plastic skull", "A plastic scary  
skull that you found in Haunted House");
```

```
Item iceCream = new(new string[] { "icecream", "vanilla icecream" }, "A vanilla  
icecream", "People say this vanilla icecream is the best");
```

```
Item miniBoat = new(new string[] { "miniboat", "red miniboat" }, "A red miniboat", "Staffs  
are giving out miniboat for the kids");
```

```
//player initial inventory
```

```
me.Inventory.Put(phone);
```

```
me.Inventory.Put(wallet);
```

```
me.Inventory.Put(bag);
```

```
bag.Inventory.Put(ring);
```

```
//locations inventories
```

```
//spawn
```

```
spawnPoint.Inventory.Put(ticket);
```

```
//ferrisWheel
```

```
ferrisWheel.Inventory.Put(miniFan);
```

```
//jungle
```

```
jungleRide.Inventory.Put(coin);
```

```
jungleRide.Inventory.Put(flower);
```

```
//haunted house
```

```
hauntedHouse.Inventory.Put(skull);
```

```
//icecream
```

```
iceCreamShop.Inventory.Put(iceCream);
```

```
//boat race
```

```
boatRace.Inventory.Put(miniBoat);
```

```
Console.WriteLine("Welcome to Swin Adventure!\nYou have arrived at the front gate of  
the theme park.");
```

```
while (true)
```

```
{
```

```
    string command = "";
```

```
    Console.Write("Command -> ");
```

```
    command = Console.ReadLine();
```

```
    string[] parts = command.ToLower().Split(' ');
```

```
    switch (parts[0])
```

```
{
```

```
    case "exit":
```

```
    case "end":
```

```
    {
```

```
        Console.WriteLine("Bye Adventurer!");
```

```
        return;
```

```
    }
```

```
    case "look":
```

```
    case "inventory":
```

```
    case "inv":
```

```
    {
```

```
        Console.WriteLine(new LookCommand().Execute(me, parts));
```

```
        break;
```

```
    }
```

```
    case "move":
```

```
    case "go":
```

```

        case "head":
        case "leave":
        {
            Console.WriteLine(new MoveCommand().Execute(me, parts));
            break;
        }
        default:
        {
            Console.WriteLine("Sorry. I don't know how to response that");
            break;
        }
    }
}
}
}
}
}

```

TestPath.cs

```

using NUnit.Framework.Legacy;
using SwinAdventure;
using Path = SwinAdventure.Path;
namespace UnitTest;
[TestFixture]
public class TestPathAndMove
{
    private Player player;
    private Location hallway;
    private Location garden;

```

```
private MoveCommand moveCommand;
```

```
[SetUp]
```

```
public void Setup()
```

```
{
```

```
    hallway = new Location("Hallway", "A long well lit hallway.");
```

```
    garden = new Location("Garden", "A lush garden with flowers.");
```

```
    // Add two-way paths
```

```
    hallway.AddPath(new Path(new string[] { "south" }, "South Path", "A path leading south.",  
garden));
```

```
    garden.AddPath(new Path(new string[] { "north" }, "North Path", "A path leading back  
north.", hallway));
```

```
    player = new Player("Alex", "an adventurer", hallway);
```

```
    moveCommand = new MoveCommand();
```

```
}
```

```
[Test]
```

```
public void TestMoveSouth()
```

```
{
```

```
    string result = moveCommand.Execute(player, new string[] { "move", "south" });
```

```
    Assert.That(player.Location, Is.EqualTo(garden));
```

```
    Assert.That(result, Does.Contain("You move south"));
```

```
    Assert.That(result, Does.Contain("Garden"));
```

```
}
```

[Test]

```
public void TestMoveNorthBack()
```

```
{
```

```
    // Move to garden first
```

```
    player.Location = garden;
```

```
    string result = moveCommand.Execute(player, new string[] { "move", "north" });
```

```
    Assert.That(player.Location, Is.EqualTo(hallway));
```

```
    Assert.That(result, Does.Contain("You move north"));
```

```
    Assert.That(result, Does.Contain("Hallway"));
```

```
}
```

[Test]

```
public void TestInvalidDirection()
```

```
{
```

```
    string result = moveCommand.Execute(player, new string[] { "move", "east" });
```

```
    Assert.That(player.Location, Is.EqualTo(hallway)); // Still in original location
```

```
    Assert.That(result, Is.EqualTo("There is no path to east"));
```

```
}
```

[Test]

```
public void TestMissingCommand()
```

```
{
```

```
string result = moveCommand.Execute(player, new string[] { "move" });
```

```
Assert.That(result, Is.EqualTo("Move where?").IgnoreCase);
```

```
}
```

```
[Test]
```

```
public void TestIdentifiablePath()
```

```
{
```

```
    Path southPath = hallway.GetPath("south");
```

```
    ClassicAssert.IsTrue(southPath.AreYou("south"));
```

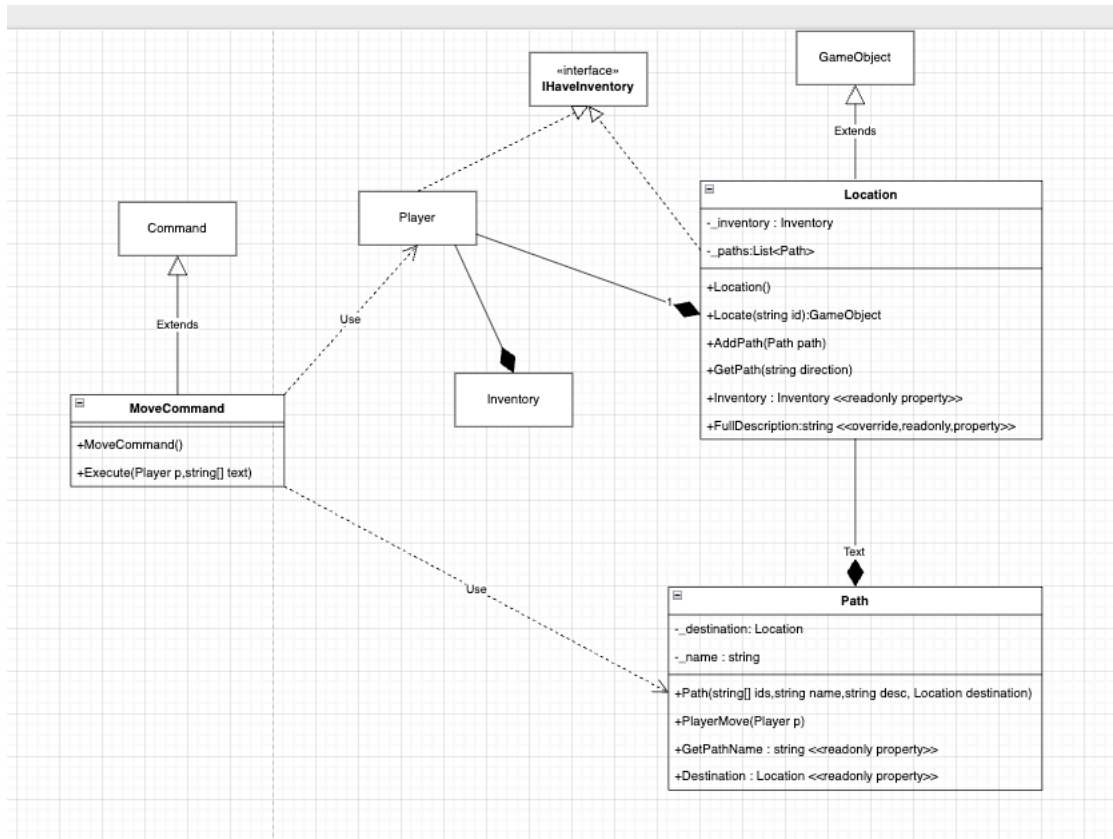
```
    ClassicAssert.IsFalse(southPath.AreYou("north"));
```

```
}
```

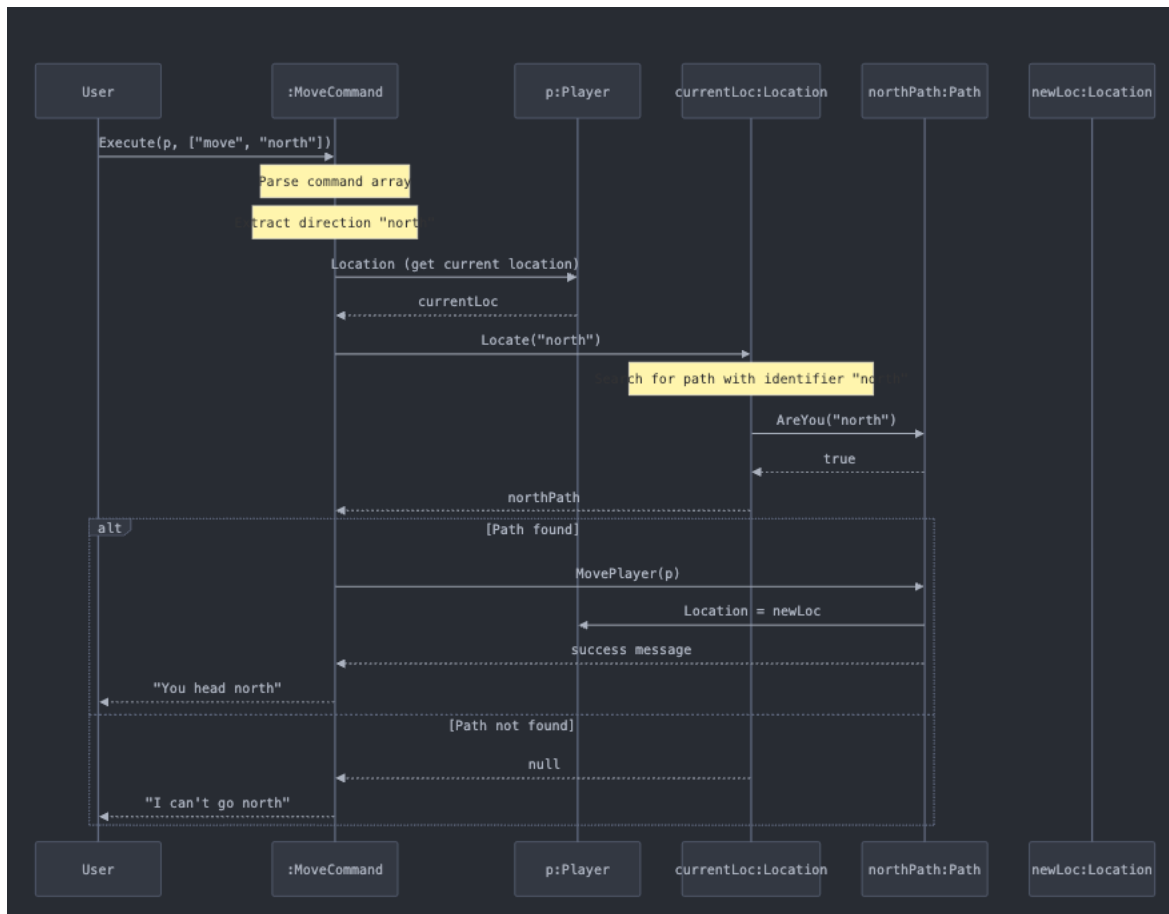
```
}
```

UML diagram





## Sequence Diagram



TestCase

TEST EXPLOR...

Filter (e.g. text, !exclude, @tag)

41/411.4s

TestIden

TestIden

38

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

TestIden

38

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

PROBLE

/Users  
del, C  
leObj